

The Rule of Three, Rule of Five, and Rule of Zero

It states: **If your class needs to explicitly define any one of the following three special member functions, it almost certainly needs to define all three.**

1. **Destructor:** `~MyClass()`
2. **Copy Constructor:** `MyClass(const MyClass& other)`
3. **Copy Assignment Operator:** `MyClass& operator=(const MyClass& other)`

Why does this rule exist?

If you do not write these three methods, the C++ compiler will kindly generate them for you. However, the compiler's default versions only perform a **shallow copy**.

If your class manages a dynamic resource, the compiler's shallow copy becomes a ticking time bomb. Here is exactly what happens if you ignore the rule:

1. **You write a Destructor:** We write a destructor, which means we are using a pointer. And using the destructor to delete the Heap pointer so we don't get a memory leak.
2. **You ignore the Copy Constructor:** You let the compiler do a shallow copy. Now, **Object A** and **Object B** share the exact same Heap pointer.
3. **The Crash (Double Free):** When **Object A** dies, your custom destructor correctly deletes the memory. But when **Object B** dies a moment later, its destructor tries to delete the exact same memory address. The system crashes with a "**Double Free**" or "**Segmentation Fault**" error.

The Rule of Five

C++11 introduced "**Move Semantics**." Because of this, the **Rule of Three** became the **Rule of Five**. If you manage memory, you should now define:

1. Destructor
2. Copy Constructor
3. Copy Assignment Operator
4. **Move Constructor**
5. **Move Assignment Operator**

The Rule of Zero (The Ultimate Goal)

This is the ultimate goal in modern C++. **We should design our classes so that we don't have to write any of the five methods.**

How? By never managing raw Heap pointers (**new/delete**) directly. If you use standard libraries (like **vector** or **string**) or **Smart Pointers** (**unique_ptr**), those objects already follow the Rule of Five internally. If your class is composed entirely of those smart objects, you can let the compiler do everything for you perfectly safely.